

DIJKSTRA + forwards Dijkstra

- shortest path  $\textcircled{S} \rightarrow \textcircled{T}$
- expand the node with smallest distance
- Time:  $O(m \log m)$
- Space  $O(m)$
- no negative cycles
- single-source
- sparse graphs

(GREEDY)  
↓  
FAST

\* backwards Dijkstra:

- distances "to t", not "from s"
- many queries ending at same t.
- Dijkstra = BFS but smarter because it uses a priority queue.

!! Fails at negative cycles

! Slow on negative costs

Input:  $G = \text{dir. graph w costs}$   
PriorityQueue  $q$  Output:  $\text{dist}[], \text{prev}[], \text{next}[]$

$q.\text{enqueue}(s, 0)$

$q.\text{enqueue}(y, \text{dist}[y])$  sum of edge costs

BFS  $O(m)$

un. of edges

- all vertices accessible from  $\textcircled{S}$
- find path from  $\textcircled{S} \rightarrow \textcircled{T}$
- min length path (shortest path)  $\textcircled{S} \rightarrow \textcircled{T}$  in terms of edges (not costs)

• Time:  $O(m+n)$  (GRAPH TRAVERSAL)

• Space: LEVELS

→ unweighted (no costs)

→ levels (BREADTH)

→ directed/undirected

→ single-source

→ general, fast graph traversal

→ cyclic graphs

!! Fails at costs

Pr. correctness:

- ① all req. vertices are access.
- ② alg. finishes
- ③ all access vert. are returned

BFS (GRAPH TRAVERSAL)

↓  
BACK (DEPTH)

• cycle detection

• topo sorting

• connected component discovery

• explore as far as possible along one path before backtracking

• Time:  $O(m+n)$  → directed graph

• Space:  $O(m)$  → structure, order, cycles, comp.

!! Fails at shortest paths in unweighted

BELLMAN-FORD

- shortest path  $\textcircled{S} \rightarrow \textcircled{T}$
- relax all edges repeatedly
- Time:  $O(m \cdot m)$  (BRUTE FORCE)
- Space:  $O(m)$
- no negative cycles
- space-optimisation of DP
- robust → negative edges ok ✓

↓  
BATE

!! Fails at negative cycles

! Slow on big graphs

\* Bellman-Ford + DP:

Shortest ✓  
cycle of  
total cost.

→ Time:  $O(m \cdot m)$

→ Space:  $O(m^2)$

→ shortest path with  $\leq k$  edges.

→ proofs (theoretical)

→ recurrence relation.

→ cycle-detection

FLOYD-WARSHALL = Bellman-Ford for all pairs.

• shortest path between all pairs

• try every node as intermediate

• Time:  $O(m^3)$

• Space:  $O(m^2)$

→ no negative cycles

→ dense graphs

→ need all pairs distances

!! Fails at negative cycles

! Overkill for single-source

→ lowest cost walk between all vertices.

TARJAN (SCC)

• 1 DFS pass

• index, lowlink, stack (1-pass)

• advanced DFS

• Time:  $O(m+n)$

• Space:  $O(m+n)$

!! Advanced.

KRUSKAL → look at all edges in the graph

→ merge components

• min spanning tree

1) sort edges by cost

2) pick cheapest edge & check cycle

3) repeat until  $n-1$  edges → tree

PRIM → look only at edges touching  $v$ .

• min spanning tree

1) don't sort edges by cost.

2) pick cheapest edge leaving tree.

→ expand current tree

Hamiltonian cycle = contains all vertices once, except for the first/last.

A\*

• shortest path  $\textcircled{S} \rightarrow \textcircled{T}$

•  $A^* = \text{Dijkstra} + \text{heuristic}$

(in the priority queue we have  $\text{dist}[x] + h(x)$  instead of  $\text{dist}[x]$ . (GREEDY))

→ no negative costs

→  $h(x)$  heuristic for input

(estimation of dist from  $\textcircled{T}$ )

• Time:  $O(m \cdot \log m)$

• Space:  $O(m)$

→ single-source

→ single-target

→ maps, geometry

!! Fails at negative costs

! wrong heuristic

PriorityQueue  $q$

$q.\text{enqueue}(s, h(s))$

$q.\text{enqueue}(y, \text{dist}[y] + h(y))$

→ simpler

→ parse-in

→ directed graph (2-pass)

→ find strongly-connected components.

• Time:  $O(m+n) \times 2$

• Space:  $O(m+n)$

!! Fails at costs

! Less efficient than Tarjan

! Needs parse-in logic

! Fails at shortest paths

TOPOLOGICAL SORT

• order vertices of a DAG s.t. for every edge  $(x, y)$ , vertex  $x$  is before vertex  $y$

→ DAG

• Time:  $O(m+n)$

→ dependencies

→ task scheduling pb.

→ detects cycles (Kahn)

→ predecessor counting (Kahn)

→ DFS-based

! if  $\exists$  cycle.

! Fails at shortest paths

! Result not unique



